



Introduction to UNIX

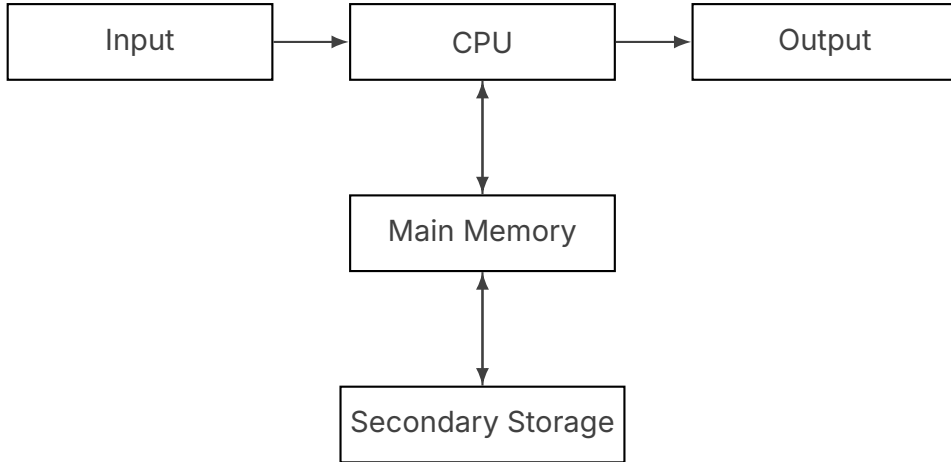
NCSC101

Prof. Pranay Kumar Saha

July 29, 2025

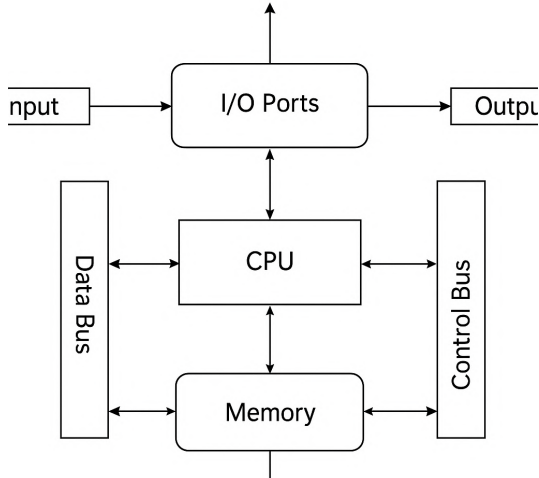


Computer





Computer Structure





What is an Operating System?

- An **Operating System (OS)** is the most important software that runs on a computer. It manages the computer's memory, processes, and all of its software and hardware.
- A control program that allocates resources and schedules tasks.
- Manages hardware such as CPU, memory, storage, and devices.
- Provides an interface for users to interact with the system.
- Enables multitasking by rapidly switching between processes.



Example

Think of it as the manager of a restaurant:

- It tells the chefs what to cook.
- It manages the pantry.
- It directs waiters to serve customers.
- It ensures everything runs smoothly without chaos.



Example

Think of it as the manager of a restaurant:

- It tells the chefs what to cook.
- It manages the pantry.
- It directs waiters to serve customers.
- It ensures everything runs smoothly without chaos.

You already use an OS every day!

- Chefs as CPU
- Pantry as Memory/RAM
- Waiters as Input/Output
- Customer as You!



The World Before UNIX

In the early days of computing (1960s), things were different:

- **One Task at a Time:** Computers could only run one program at a time ("batch processing").
- **One User at a Time:** A single person controlled the entire machine.
- **Not Portable:** Software written for one type of computer would not work on another.
- **Extremely Expensive:** Computers were massive machines owned by large corporations and universities.



The World Before UNIX

In the early days of computing (1960s), things were different:

- **One Task at a Time:** Computers could only run one program at a time ("batch processing").
- **One User at a Time:** A single person controlled the entire machine.
- **Not Portable:** Software written for one type of computer would not work on another.
- **Extremely Expensive:** Computers were massive machines owned by large corporations and universities.

There was a need for a simpler, more efficient way to work.



What is UNIX?

Definition

UNIX is a powerful, multi-user, multi-tasking operating system originally developed in the 1970s. It's known for its portability, flexibility, and a powerful command-line interface.

- **Multi-User:** Allows multiple users to access the system and its resources simultaneously.
- **Multi-Tasking:** Allows a single user to run multiple programs at the same time.
- **Portability:** Can be run on a wide variety of hardware platforms, from personal computers to supercomputers.



The UNIX Philosophy

The design of UNIX is guided by a set of core principles that contribute to its power and longevity.

- **Write programs that do one thing and do it well.**
 - A tool for sorting should only sort. A tool for counting words should only count words.
- **Write programs to work together.**
 - The output of one program should be the input of another. This is like an assembly line.
- **Write programs to handle text streams, as that is a universal interface.**
- Everything is a file.

Think of them as **Lego blocks**. Each block is simple, but you can combine them in infinite ways to build something complex and powerful.



The Birth of Unix

- **Where:** Bell Labs (the research arm of AT&T).
- **When:** Late 1969 / Early 1970s. **Who:** Ken Thompson and Dennis Ritchie (and others).
- **The Story:** They wanted to create a simple, multi-user, multi-tasking environment to run a game called "Space Travel" on a spare, weak computer.
- **Key Breakthrough:** They later rewrote Unix in a new programming language they created called 'C'. This made Unix portable – a revolutionary idea!



The Origins of UNIX

- Developed at **AT&T's Bell Labs** in the late 1960s and early 1970s.
- Key creators include **Ken Thompson**, **Dennis Ritchie**, and others.
- Initially written in assembly language, it was later rewritten in the **C programming language**.
- This rewrite in C was revolutionary, making UNIX highly portable.



Figure: Ken Thompson and Dennis Ritchie

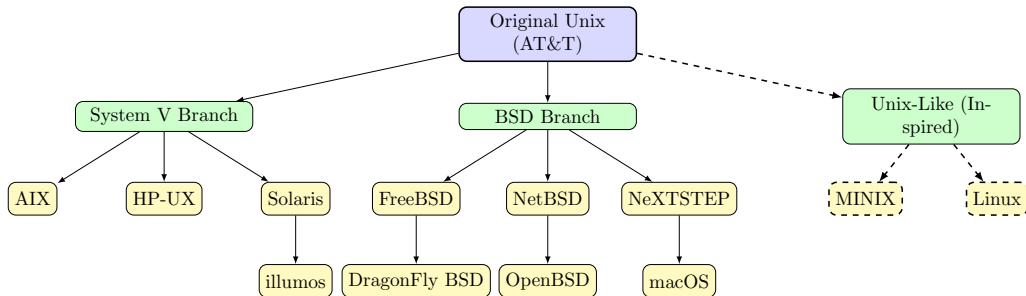


The Evolution and Family Tree

- **System V:** One of the first commercial versions from AT&T.
- **BSD (Berkeley Software Distribution):** Developed at the University of California, Berkeley. It introduced many new features.
- **Modern Descendants:**
 - **Linux:** A UNIX-like kernel developed by Linus Torvalds. Not technically UNIX, but follows its design principles.
 - **macOS:** Based on the Mach kernel and BSD. It is a certified UNIX system.
 - **Solaris:** Developed by Sun Microsystems (now Oracle).
 - **AIX:** Developed by IBM.



Family Tree





UNIX vs. Linux: What's the Difference?

UNIX

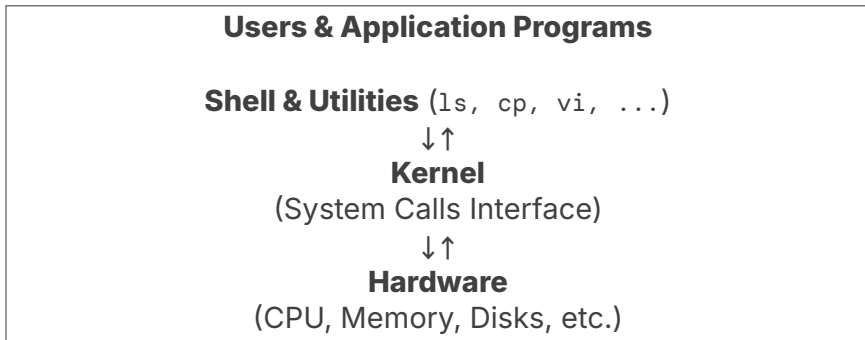
- A family of operating systems that meet a certain specification (The Single UNIX Specification).
- Some versions are proprietary and cost money (e.g., AIX, Solaris).
- macOS is a certified UNIX.

Linux

- Technically, Linux is just the kernel. A "Linux distribution" (like Ubuntu, Red Hat) includes the kernel plus a vast collection of software.
- It is a UNIX-like system, not a certified UNIX. It was built from scratch to behave like UNIX.
- It is open-source and free.



The Layered Architecture of UNIX



The UNIX system is primarily composed of three main parts: the Kernel, the Shell, and the Utilities/Tools.



Component 1: The Kernel

The Core of the OS

The Kernel is the heart of the operating system. It is a master control program that manages the system's resources.

- Interacts directly with the hardware.
- Manages memory (allocating it to processes).
- Schedules CPU time for different tasks.
- Handles file storage and retrieval.
- Manages system calls from the shell and other programs.
- The user does not interact directly with the kernel.



What the Kernel Does

The kernel has complete control over everything in the system and is responsible for several critical tasks:

- **Process Management:** It decides which programs get to use the central processing unit (CPU) and for how long. This is what allows your computer to multitask, running several programs at the same time.
- **Memory Management:** The kernel manages the computer's memory (RAM), allocating it to different programs as needed and ensuring that one program doesn't interfere with another's memory space.



What the Kernel Does

- **Device Management:** It manages communication between software and hardware devices like your keyboard, mouse, monitor, and printer. It does this through programs called device drivers.
- **System Calls and Security:** When an application needs to perform a task that requires access to hardware, it makes a "system call" to the kernel. The kernel then securely performs the requested action, acting as a gatekeeper to protect the system's resources.



Component 2: The Shell

The Command Interpreter

The Shell is the interface between the user and the kernel. It's a command-line interpreter that reads user commands and asks the kernel to execute them.

- It's a program that runs when you log in.
- Provides a prompt (e.g., \$ or %) waiting for input.
- Translates human-readable commands into something the kernel can understand.
- It is also a powerful scripting language.



Types of Shells

There are many different shells, each with slightly different features and syntax.

Shell Name	Common Abbreviation
Bourne Shell	sh
C Shell	csh
Korn Shell	ksh
Bourne Again Shell	bash
Z Shell	zsh

Table: bash is the most common default shell on many UNIX-like systems today, especially Linux.



Component 3: Utilities and Applications

The Toolbox

These are the thousands of programs that provide the functionality of the UNIX system. They are often referred to as "commands".

- Range from simple tools to complex applications.
- Examples:
 - `ls` - list files
 - `cp` - copy files
 - `mv` - move/rename files
 - `grep` - search for text
 - `vi`, `emacs` - text editors
 - Compilers (`gcc`), web servers, databases, etc.



Hierarchical Structure

Everything Starts at the Root

The UNIX file system is organized as a single hierarchical tree structure, starting from the root directory, denoted by `/`.

- All files and directories are located under this single root.
- This includes physical devices like hard drives, CD-ROMs, and USB drives, which are "mounted" into the file system tree.
- Example path: `/home/user/documents/report.txt`



Key Directories

A standard set of directories exists at the top level, each with a specific purpose.

- `/bin` - Essential user command binaries
- `/etc` - System configuration files
- `/home` - User home directories
- `/lib` - Essential shared libraries
- `/dev` - Device files (representing hardware)
- `/tmp` - Temporary files
- `/usr` - User programs and data
- `/var` - Variable files (logs, caches)



File Types in UNIX

In UNIX, almost everything is treated as a file.

- **Regular Files (-):** Contain data, such as text, images, or program instructions.
- **Directories (d):** Files that contain a list of other files.
- **Symbolic Links (l):** A pointer to another file.
- **Device Files (c, b):** Represent hardware devices.
 - **Character (c):** For devices that handle data character by character (e.g., terminals).
 - **Block (b):** For devices that handle data in large blocks (e.g., hard drives).



Key Features Driving Popularity

- **Portability:** Written in C, it can be adapted to new hardware with relative ease.
- **Powerful Shell:** The command-line interface is a rich programming environment, allowing for complex tasks to be automated.
- **Pipes and Filters:** The ability to chain commands together (`command1 | command2`) is incredibly powerful.
- **Minimalist and Modular Design:** Small tools that do one thing well can be combined to perform complex operations.
- **File System Abstraction:** The "everything is a file" concept simplifies interaction with devices and processes.



More Features

- **Security and Permissions:** A robust model for file permissions (read, write, execute) for owner, group, and others.
- **Stability and Reliability:** UNIX systems are known for their ability to run for long periods without crashing.
- **Open Standards:** Many UNIX concepts have become part of open standards like POSIX, ensuring compatibility across systems.



Advantages of UNIX

- **High Security:** Robust permission system protects the OS from malicious programs.
- **Portability:** Can run on a vast range of hardware.
- **Powerful Scripting:** The shell allows for extensive automation of tasks.
- **Low Resource Usage:** Generally requires less memory and CPU compared to other graphical operating systems.
- **Stability:** Proven track record of reliability, especially in server environments.



Disadvantages of UNIX

- **Steep Learning Curve:** The command-line interface can be intimidating for new users.
- **Less User-Friendly (Traditionally):** The focus is on power and flexibility, not always on a polished graphical user interface (GUI).
- **Cryptic Commands:** Many commands have short, non-intuitive names (e.g., `grep`, `awk`, `sed`).
- **Limited Hardware Support (for some versions):** Some proprietary UNIX versions may only support specific hardware.



Summary

- UNIX is a foundational, multi-user, multi-tasking operating system.
- Its architecture is layered: **Hardware → Kernel → Shell → Applications**.
- The **Kernel** manages resources, and the **Shell** is the user's command interface.
- The "everything is a file" philosophy and powerful command-line tools are its defining features.
- Its principles have heavily influenced modern computing, most notably in **Linux** and **macOS**.



Introduction to UNIX

Thank You for Listening!